

Lab 5: Pick and Place*

EECS/ME C106A/206A Fall 2026

Goals

Over the course of this lab you will:

- Filter and Process Pointcloud messages in Ros2
 - Use MoveIt to compute inverse kinematics solutions
 - Combine Perception and Planning to have the UR7e pick up and place a block
 - Use HSV color thresholding to find a target, and place the block on it
-

Helpful Documentation:

- UR7e SDK: https://docs.universal-robots.com/Universal_Robots_ROS2_Documentation/
- tf2 Tutorials: <https://docs.ros.org/en/humble/Tutorials/Intermediate/Tf2/Tf2-Main.html>
- PointCloud msgs: https://docs.ros.org/en/noetic/api/sensor_msgs/html/msg/PointCloud2.html
- point_cloud2: https://github.com/ros2/common_interfaces/tree/humble/sensor_msgs_py

Contents

0	Initial Setup	2
1	Introduction	3
1.1	Code Overview	3
2	Processing Perception Data	3
2.1	Visualizing the Point Cloud	4
2.2	Coordinate Transforms	4
2.3	Filtering the Point Cloud	6
2.4	How do I tune?	6
2.5	Calculating the Cube's Position	6
3	Introduction to MoveIt	8
3.1	What MoveIt Does for Us	8
3.2	Inverse Kinematics	8
3.3	Additional Resources	9

*Developed by Daniel Municio, Ishan Dogra, Satwik Chappidi, and Karim El-Refai. Additional content by Ishan Dogra and Satwik Chappidi. Updated by Ansh Gandhi, Spring 2026. Updated by Kush Hari and Purva Marfatia, Fall 2026.

4	Motion Execution via ROS2 Action Servers	9
4.1	Overview	9
4.2	Action Clients	9
4.3	Code Architecture: UR7e_CubeGrasp Node	9
4.3.1	Key Components	9
4.3.2	Execution Pipeline	10
4.4	Student Task	10
4.5	Execution Instructions and Safety	11
5	Placing the Cube on the Tape	12
5.1	HSV Color Thresholding	12
5.2	Colors in the Point Cloud	12
5.3	Placing the Cube	13
6	Appendix	14

0 Initial Setup

As usual, every step of the setup is required. Please follow carefully. Enter the Distrobox and pull the Lab 5 starter code from the **starter** remote, the same way you did in Lab 3:

```
cd ~/ros_workspaces
git fetch starter
git restore --source=starter/main --staged --worktree -- lab5
```

If you haven't already done so in Lab 6, we will make a **.gitignore** file in our repository, which tells Git which files and directories should not be tracked or committed to the repository. Create a file titled **.gitignore** in the **~/ros_workspaces** folder and copy the following into it:

```
# ROS 2 / colcon
**/build/
**/install/
**/log/
.colcon/

# Python
**/__pycache__/
*.py[cod]
*.egg-info/
.pytest_cache/

# IDEs / editors
.vscode/
.idea/

# OS files
.DS_Store
Thumbs.db

# Python virtual environments
.venv/
venv/
env/
```

The reason we especially need this is because the **build**, **install**, and **log** directories contain automatically generated files that should never be committed. If you accidentally commit them, they can create **merge conflicts later**.

Next run:

```
git rm -r --cached --ignore-unmatch -- '**/build/**' '**/install/**' '**/log/**'
git add .gitignore
git commit -m "Add Lab 5 starter code"
git push origin main
```

1 Introduction

In Lab 3, you studied **forward kinematics**: given joint angles, compute transforms between frames on the manipulator. We are often more interested in the **inverse kinematics** problem: find joint angles that place a link at a desired pose in $SE(3)$. In this lab, you will use the Intel Realsense D435i to estimate a block pose, apply inverse kinematics to guide the UR7e to it, then grasp and place the block.

Inverse kinematics gives a target joint configuration but not a path. **Path planning** computes a constraint-respecting joint trajectory between a start and goal configuration (as you used for a basic pick-and-place). This lab, combined with lab 7 in the next module, culminates in **full-stack robotics**: combining perception, planning, and control to build a complete manipulation pipeline.

1.1 Code Overview

The starter code for this lab is more complicated than usual, so we will provide a brief overview.

The primary files you will be interacting with are listed below:

```
lab5
  src
    perception
      perception
        process_pointcloud.py
        hsv_tuner.py
    planning
      launch
        lab5_bringup.launch.py
      planning
        ik.py
        main.py
        static_tf_transform.py
```

2 Processing Perception Data

The Intel RealSense D435i has two D430 camera modules, an IR projector, an RGB module, and an IMU, which allow it to collect a variety of data.

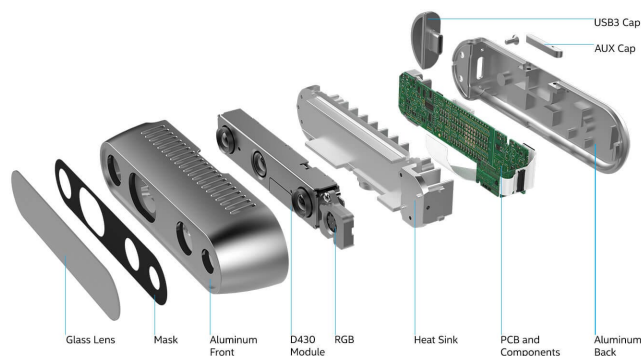


Figure 1: Breakdown of a standard RealSense camera

The camera can be launched with a variety of default publishers and parameters. In this case, we will use the PointCloud publisher. The camera uses stereo vision with its two D430 modules to create a point cloud message. To begin publishing, run:

```
ros2 launch realsense2_camera rs_launch.py pointcloud.enable:=true \
  rgb_camera.color_profile:=1920x1080x30
```

This publishes a point cloud message to the topic: `/camera/camera/depth/color/points`.

2.1 Visualizing the Point Cloud

Open a new terminal and launch RViz2:

```
rviz2
```

In RViz2:

1. Change the **Fixed Frame** to `camera_depth_optical_frame`.
2. Click **Add** → **By Topic** → `/camera/camera/depth/color/points`.

You should now see the point cloud of the field in front of the camera. This point cloud is generated using *stereopsis*, where two cameras (in this case, infrared cameras) a known distance apart from each other, take an image of the same scene and use the difference between corresponding points in different images to calculate the distance or "depth" from each point to the camera. We've seen point clouds before in Lab 4 from the LiDAR system on the TurtleBots. Rotating LiDAR systems cover a wider angular range and can capture an entire scene. However, most LiDAR point clouds do not include full color like this one and instead store intensity values based on reflected light.

2.2 Coordinate Transforms

We currently have the point cloud in the frame `camera_depth_optical_frame` but want to analyze that data in the `base_link` frame. You cannot directly call `tf2_buffer.lookup_transform` between these two frames because they are not part of the same TF tree (TF2 has no idea how they are related).

The key missing link is the fixed transform between the camera and the robot. The transformation $G_{depth \rightarrow wrist.3}$ remains constant throughout the robot's motion (the 3D printed mount does not change shape). If we add this static transform to TF2, then we can compute $G_{depth \rightarrow base_link} = G_{depth \rightarrow wrist.3} \cdot G_{wrist.3 \rightarrow base_link}$ and transform points from `camera_depth_optical_frame` into `base_link`.

Refer to Figure 2 above for a diagram of the relevant coordinate frames needed to implement this transformation.

TF Broadcaster (complement to TF Listener) In Lab 3 you wrote a TF listener, which uses a `Buffer` and `lookup_transform` to *read* transforms that are already in the TF tree. You will use a listener again later in this lab to get the point cloud into the robot's frame. For this task we need a *TF Broadcaster* instead, which *publishes* a transform to the `/tf` (dynamic) or `/tf_static` (static, latched) topics so that it becomes part of the TF tree.

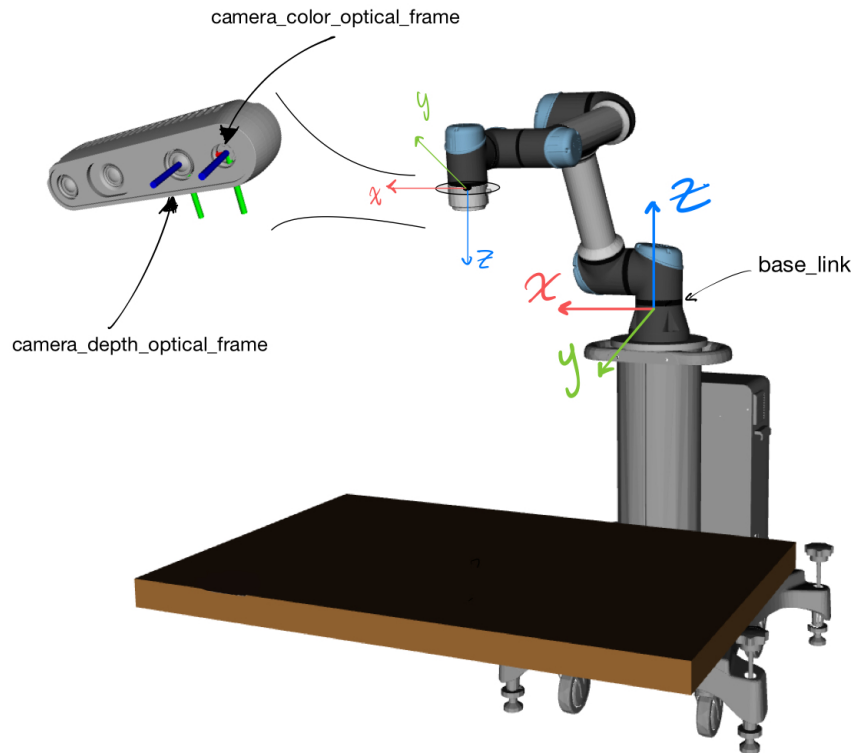


Figure 2: Coordinate Frames

- **Broadcaster:** announces a transform by sending a `TransformStamped` with `header.frame_id` (parent), `child_frame_id` (child), translation, and rotation.
- **Static vs Dynamic:** use `StaticTransformBroadcaster` for fixed geometry (latched on `/tf_static`); use `TransformBroadcaster` with a timer for time-varying transforms (on `/tf`).

In this task, you will *broadcast* a static transform from the RealSense depth camera frame to `wrist.3.link`. This connects the camera-side measurement chain to the robot base (see Appendix 6).

Note on starter code: The provided skeleton creates a `StaticTransformBroadcaster`; you will fill in the `TransformStamped` fields (parent/child frames, translation, quaternion (look into `scipy.spatial.transform`)) and publish at a fixed rate.

Task 1: Implement the missing code in `static_tf_transform.py` to create a TF node that will continuously publish the static transformation between the `camera_depth_optical_frame` and the `wrist.3.link` frame. The provided skeleton creates a `StaticTransformBroadcaster`; you will fill in the `TransformStamped` fields (parent/child frames, translation, quaternion (look into `scipy.spatial.transform`)) and publish at a fixed rate. **Hint:** Refer to Figures 5 and 6 in the Appendix 6 for a visual of what this looks like.

To verify your solution, you can visualize your TF2 tree! First, make sure you're running your broadcaster and the UR7e arm.

```
ros2 run ur7e_utils enable_comms
```

```
ros2 run planning tf
```

Then, run the following command, which will have `tf2` listen to all frames being published over 5 seconds.

```
ros2 run tf2_tools view_frames
```

Once this command finishes running, it will save the graph to a file with a name along the lines of `frames_2026_date_time_stuff.pdf`, which you can open as a PDF with the file explorer, or with

Checkpoint 1

Submit a checkoff request at <https://tinyurl.com/ee106a-fa261ab>. At this point you should be able to:

- Show the point cloud you visualized in Rviz
- Show your connected TF tree
- Explain how your static transform broadcaster works and how it allows you to connect the 'chain' mentioned in Task 1

2.3 Filtering the Point Cloud

Now that we have a point cloud from the camera and know the transform between the camera frame and base link, we need to filter points that are not of interest (table, lab PC, background objects, etc).

Task 2: To filter these noisy points, we first need to transform the point cloud from the camera frame to the base link frame. Implement this logic in `process_pointclouds.py`.

Now that we have the point cloud in the `base_link` frame, we can now "crop" the point cloud by **discarding points outside a certain XYZ range**. We have provided the minimum and maximum Z values, as well as the maximum Y value, in `process_pointcloud.py`. Remember, since you transformed the points, the Z and y axes here correspond to the `base_link` frame in Figure 2 above (can also be seen in rviz). You may tune them as needed.

2.4 How do I tune?

We've written helper functions that will visualize the planes that you're using to filter out the pointcloud – you can add the `filter_planes` topic, which is of type `MarkerArray`, so that you can see where the planes you're filtering by are.

Thanks to the way we've declared the min and max values as ROS2 parameters, you DO NOT have to rebuild and source every time you change a value! Use `ros2 param set /process_pointcloud min_z -0.14` to change the values, and if you want to see what they currently are use `ros2 param dump /process_pointcloud` :)

Task 3: Implement the filtering logic in the function `pointcloud_callback` in `process_pointcloud.py`. Your code should filter points that:

- Are outside the block plane
- Are far from the robot (optional)

To verify that your filtered point cloud isn't "overcropped" (it doesn't have any noisy points but still has points to represent the cube), add the following topic in RViz2:

- `/filtered_points`: Shows the filtered point cloud

2.5 Calculating the Cube's Position

Task 4: Calculate the cube's position and height from the point cloud. For x and y , you may simply compute the mean or median of the filtered points (or something more sophisticated).

To grasp the cube well, the robot needs to know how tall it is, so do not hardcode that either:

- The **top** of the cube is about the height of the highest filtered points. A high percentile (like the 95th) is less noisy than the maximum.

- The **table** height is the height of the points just outside the cube. Use the points (from all of `points_base`, not just the filtered ones) that are between `TABLE_RING_MIN` and `TABLE_RING_MAX` away from the cube's center in x and y .

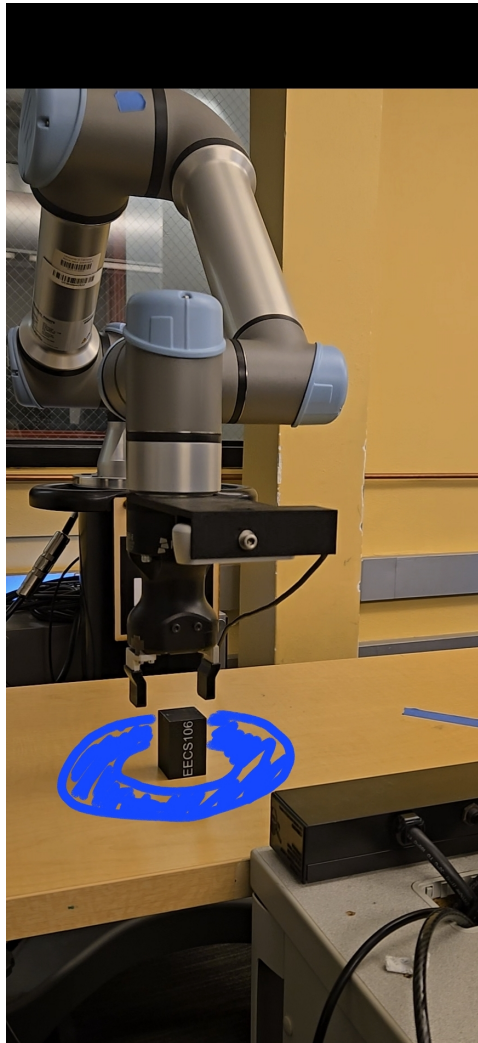


Figure 3: Visual depiction of what the table ring is

The starter code then computes the cube's height (top minus table), publishes it on `/cube_height`, and sets the z of `/cube_pose` to the middle of the cube. For this to work, `max_z` has to be above the top of your cube, so the top face makes it through the filter. If it isn't, the cube will look shorter than it really is, and `process_pointcloud` will print a warning telling you to raise `max_z`.

Do not forget to add a header to your PointStamped msg with a frame ID and timestamp!

Add the following topic to RViz2 to visualize the cube's position:

- `/cube_pose`: Shows the cube's detected position

You should now be able to see the cube's detected position exactly at the cube's real location, halfway up the cube. You can check the height with `ros2 topic echo /cube_height` and a ruler.

Checkpoint 2

Submit a checkoff request at <https://tinyurl.com/ee106a-fa261ab>. At this point you should be able to:

- Show that you have completed the prelab

- Confirm that the cube's position is correctly detected in Rviz
- Explain your filtering code
- Explain how you could find the table height from just the pointcloud? What would the plane look like if you did a best-plane fit of the pointcloud? How could RANSAC help?

3 Introduction to MoveIt

You already used MoveIt in Lab 3, where the `/compute_cartesian_path` service planned straight-line paths for your action server. In this lab you will use two other parts of it: inverse kinematics (`/compute_ik`), to find the joint angles that put the gripper at a pose, and joint-space motion planning (`/plan_kinematic_path`), to get a collision-free trajectory to those joint angles.

3.1 What MoveIt Does for Us

In Lab 3 you wrote forward kinematics yourself: given the joint angles, find where the wrist is. To pick up the cube we need to go the other way, which is **inverse kinematics**: given where we want the gripper to be, find joint angles that put it there. IK is a lot harder than FK. A pose can have several solutions (elbow up or elbow down, for example) or none at all, and for most arms there is no simple formula, so MoveIt solves it numerically.

Once we have the goal joint angles, we still need a way to get there without hitting the table or the robot itself. MoveIt plans that path with a sampling-based planner (RRT-Connect, which is what `plan_to_joints()` asks for), then times it so the joints stay within their velocity and acceleration limits.

Each of these pieces (IK solvers, collision checking, motion planners, trajectory timing) is a big topic on its own, and you are welcome to write your own versions for your final project. For this lab, using MoveIt lets us focus on putting perception and planning together.

3.2 Inverse Kinematics

The provided `ik.py` script demonstrates how to interface directly with MoveIt through ROS 2 service calls. In particular, the script:

- Builds and sends a `PositionIKRequest` to the `/compute_ik` service to compute a valid joint configuration for a given end-effector pose.
- Extracts the returned `JointState` from the IK response.
- Constructs a motion planning request and sends it to the `/plan_kinematic_path` service using MoveIt's planning interface.
- Returns a full motion trajectory that the robot can execute.

Building these requests by hand, instead of using a higher-level helper, shows you exactly what MoveIt needs to know to solve IK and plan a path.

Task 5: Within `ik.py`, complete the `compute_ik` implementation according to the comments provided in the starter code. You will likely need to look up the format of these message types to accomplish this.

- **IMPORTANT** Once you have added your implementation, please test that your IK passes the unit tests in the main function of the node. Do this by running

```
ros2 launch ur_moveit_config ur_moveit.launch.py ur_type:=ur7e launch_rviz:=true
```

in a new terminal to start MoveIt and then run `ros2 run planning ik` and ensure that you see the successful log in terminal.

- Once the implementation is tested and working, add this node to the `lab5.bringup.py` launch file (**note: you can set the `name=` parameter to pretty much anything, you're not going to be updating any**

parameters in real time) so that all necessary nodes can be launched at once. Add IK to the end of the launch description so that the transforms and MoveIt are all running before we make an IK call.

3.3 Additional Resources

- MoveIt Documentation: <https://moveit.picknik.ai/humble/index.html>
- MoveIt Concepts Overview: <https://moveit.picknik.ai/humble/doc/concepts/concepts.html>

4 Motion Execution via ROS2 Action Servers

4.1 Overview

In this lab, we separate the motion *planning* and *execution* stages. Although MoveIt is capable of both planning and executing trajectories, we deliberately restrict MoveIt to only provide joint trajectories. Execution is then performed directly through the Universal Robots controller interface using the `FollowJointTrajectory Action Server`.

4.2 Action Clients

You already know how actions work from Lab 3, where you wrote the `MoveStraight` action *server*. A client sends a goal, the server accepts or rejects it, sends feedback while it works, and finishes with a result. This time `main.py` is the action *client*, and the server is the UR's trajectory controller, which runs the `control_msgs/action/FollowJointTrajectory` action. Its goal is a `JointTrajectory`, the same message you built for direct joint control in Lab 3, and its result tells you whether the robot finished the trajectory.

Using an action from the client side takes three steps, and each one is asynchronous:

1. Create an `ActionClient` for the action's name (here `/scaled_joint_trajectory_controller/follow_joint_trajectory`) and wait for the server with `wait_for_server()`.
2. Send the goal with `send_goal_async()`. This returns a future, which finishes once the server has accepted or rejected the goal. In `main.py`, `_on_goal_sent()` runs when it finishes and checks that the goal was accepted.
3. If it was accepted, call `get_result_async()` on the goal handle. This returns a second future, which finishes once the robot is done moving. `_on_exec_done()` runs when it finishes and starts the next job in the queue.

Attaching callbacks instead of waiting on the futures means the node keeps running (and receiving joint states) while the robot moves.

4.3 Code Architecture: UR7e_CubeGrasp Node

The file `main.py` defines a node that:

- subscribes to the cube pose and robot joint state
- plans to waypoints via the provided `IKPlanner`
- enqueues grasp and placement sub-tasks
- executes motion via the `FollowJointTrajectory Action Server`

4.3.1 Key Components

- **Subscriptions:**
 - `/cube_pose`: cube location in base frame (the middle of the cube)
 - `/cube_height`: how tall the cube is
 - `/tape_pose`: where the blue tape is (used in Section 5)
 - `/joint_states`: current UR arm configuration
- **Planner:** `IKPlanner.compute_ik()` and `plan_to_joints()` call MoveIt's IK and motion planning services

- **Executor:** FollowJointTrajectory action client sends trajectories directly to the UR controller
- **Gripper:** toggled via a `std_srvs/Trigger` service call

4.3.2 Execution Pipeline

1. Cube pose received $\Rightarrow$ construct a job queue
2. For each queued step:
 - (a) compute IK and motion plan (MoveIt)
 - (b) send trajectory goal to UR controller (Action Server)
 - (c) wait for the result (`_on_exec_done`)
 - (d) advance to next queue entry

4.4 Student Task

For the rest of this lab, the robot starts wherever you leave it. You will free drive the arm so that the wrist camera is looking down at the table, and then start `main.py`. When the node starts up, it saves the arm's joint angles in `self.start_joints`. From there, it should pick up the cube, bring it back to that starting pose, look for the blue tape, and put the cube down on the tape. This section covers the first half (picking up the cube), and Section 5 covers the second half.

The heights in this part all come from the cube you measured. `/cube_pose` is the middle of the cube, so its top is $z + h/2$ and the table is at $z - h/2$, where h is `/cube_height`. Remember that the IK target is `wrist_3_link`, not the fingertips: the fingertips are `GRIPPER_LENGTH` below it, so add that to wherever you want the fingertips to be. **Measure GRIPPER_LENGTH on your gripper** (from the wrist flange to the fingertips) before running anything, since it is what keeps the fingers out of the table.

Task 6: Populate the queue in `cube_callback()` to perform:

1. Pre-grasp above the cube: fingertips `PRE_GRASP_CLEARANCE` above the top of the cube
2. Lower to grasp: fingertips at the middle of the cube, but never closer than `TABLE_CLEARANCE` to the table. Save how far above the bottom of the cube the fingertips are in `self.grasp_height`.
3. Close gripper ("`toggle_grip`")
4. Retreat to pre-grasp
5. Go back to the start pose (`self.start_joints`)
6. Look for the tape ("`find_tape`")

Each waypoint must be expressed as a target `JointState`, planned via `IKPlanner`, and executed through the Action Server. The start pose is already a `JointState`, so you do not need IK for that one. The "`find_tape`" job is given: it waits a second for the arm to settle at the start pose, and then calls `plan_place()` with the next tape position that comes in on `/tape_pose`. You will write `plan_place()` in Section 5.

The image below might be useful in thinking about what you should set your IK target to be:

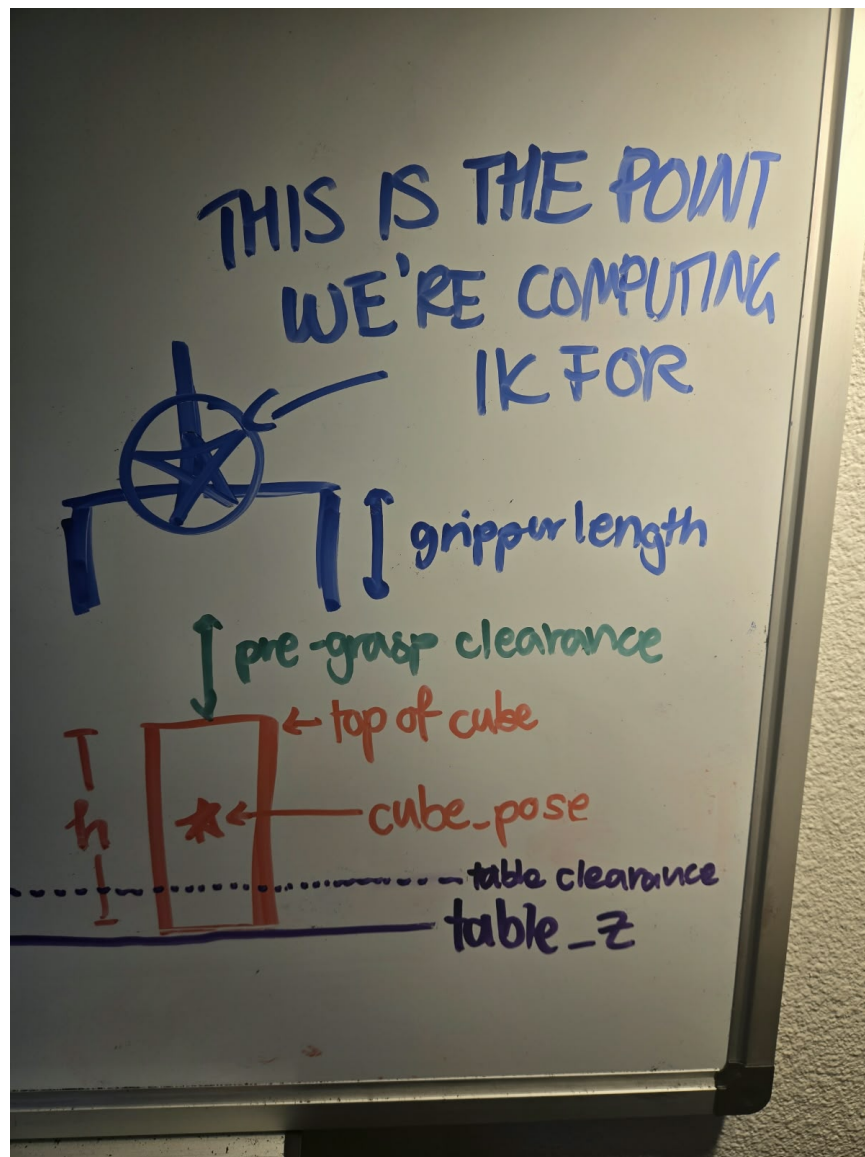


Figure 4: Diagram of lengths related to grasp

4.5 Execution Instructions and Safety

Before running: Ensure all relevant nodes are active and you don't have multiple instances of the same node running in different terminals. For example, this could happen if you already had the camera node running from checkpoint 1 and then ran the launch file. If everything has been added to your launch file correctly, all you should have to run before running `ros2 run planning main` is `enable_comms` and `lab5_bringup.launch.py`.

Set the start pose: Put the arm in freedrive like you did in Lab 3:

```
ros2 run ur7e_utils freedrive
```

Move the arm slowly and gently so that the wrist camera is looking down at the table and can see both the cube and the blue tape, then `Ctrl+C freedrive`. Check in RViz that `/cube_pose` shows up on the cube. `main.py` switches back to the trajectory controller on its own when it starts, so you do not need to do anything else after free driving.

Run the grasp node:

```
ros2 run planning main
```

Until you finish Section 5, the robot should pick up the cube, come back to the start pose and then wait for the tape.

Safety Notice: Keep your hand on the **emergency stop** at all times! Do not execute unless workspace is clear and transforms are verified.

Toggling the gripper: Make sure that when you run the grasp node your gripper starts in the open position! If you miss the cube and want to try again, you MUST run

```
ros2 service call /toggle_gripper std_srvs/srv/Trigger {}
```

in order to toggle the gripper. Failure to do so will result in the closed gripper fingers "stabbing" into the cube (assuming you set the target height correctly), damaging it.

5 Placing the Cube on the Tape

Now that the robot can pick up the cube, we want it to put the cube down somewhere specific: on a piece of blue tape on the table. To do that, we need to find the tape.

5.1 HSV Color Thresholding

The easiest way to find the tape is by its color. You could try to do this directly with RGB values, but the RGB values of the same piece of tape change a lot depending on the lighting and shadows. **HSV** (hue, saturation, value) separates the color from how bright it is:

- **Hue** is the color itself (red, yellow, green, blue, ...), as an angle around the color wheel.
- **Saturation** is how strong the color is. Gray, white and black have a saturation near 0.
- **Value** is how bright it is. Black is 0.

A shadow on the tape mostly lowers its value but does not change its hue much, so we can find the tape by looking for points with a blue hue and a high enough saturation and value. In OpenCV, hue goes from 0 to 179 (half the usual 0 to 360 degrees so it fits in a byte), and saturation and value go from 0 to 255. Blue is roughly $H \in [100, 130]$.

5.2 Colors in the Point Cloud

You might expect us to find the tape in the camera's color image. But then we would have a pixel, not a 3D position, and if you look at your TF tree, only `camera_depth_optical_frame` is connected to the robot. The color camera's frame is not.

Instead, we use the point cloud we already have. `/camera/camera/depth/color/points` has a color for every point, taken from the color camera, so we can threshold the colors of the points directly. Every point that passes already has a position in `base_link`, and the centroid of those points is where the tape is. The starter code reads these colors for you in `point_colors_hsv()` (the RealSense packs each color into a single number, so the code unpacks it into R, G and B).

Task 7: Finish the Part 5 TODOs in `process_pointcloud.py`:

- In `point_colors_hsv()`, convert the colors to HSV with `cv2.cvtColor`. The colors are in RGB order, not the BGR order OpenCV usually uses. Keep in mind that what we want to do is just change the color values, so we can reshape our array to be a $N \times 1 \times 3$ image (because `cvtColor` does expect an image!) and then shape it back to what it was after OpenCV does the conversion.
- In `pointcloud_callback()`, add a filter so the cube only uses dark points ($V \leq \text{cube_max_v}$).
- In `find_tape()`, keep the points in the tape's HSV range, and publish their centroid on `/tape_pose` as a `PointStamped`.

The HSV range for the tape depends on the lighting and on your tape, so we give you a tool to find it:

```
ros2 run perception hsv_tuner
```

This opens a window with the camera image and the mask for the current thresholds. Move the sliders until only the tape shows up white, then press **p** to print the `ros2 param set` commands that give those values to `process_pointcloud`. Once it works, you can change the defaults in `process_pointcloud.py` so you do not have to set them every time.

Add `/tape_points` and `/tape_pose` in RViz to check that you are finding the tape and not something else that is blue.

5.3 Placing the Cube

Task 8: Fill in `plan_place()` in `main.py`. It gets called with the tape's position once the robot is back at the start pose, and it should add these jobs to the queue:

1. Move above the tape (same height as the pre-grasp)
2. Lower the cube down onto the tape
3. Open the gripper ("`toggle_grip`")
4. Move back up
5. Go back to the start pose

To figure out how low to go, notice that the tape is flat on the table, so the z of `/tape_pose` is the height of the table where you want to put the cube. You are holding the cube with the fingertips `self.grasp_height` above its bottom, so if the fingertips go to that height above the tape, the bottom of the cube will be right on the table. Let go `TABLE.CLEARANCE` above that, so the cube does not get pushed into the table. When moving above the tape, make sure the bottom of the cube clears the table too. The whole cube height plus some clearance is plenty.

Run everything the same way as before: free drive to a start pose where the camera can see the cube and the tape, then run `ros2 run planning main`. The cube should end up on the tape and the arm should come back to where it started.

Safety Notice: Keep your hand on the **emergency stop** at all times!

Checkpoint 3

Submit a checkoff request at tinyurl.com/ee106a-fa26lab for a staff member to come and check off your work. At this point you should be able to:

- Demonstrate the UR7e picking up the cube and placing it on the tape, starting from a start pose you free drove to. Show it working again with the cube and the tape moved to new spots.
- Show `/filtered_points` and `/tape_points` in RViz and explain your HSV thresholds
- Explain why we needed to use HSV for the tape detection, and why don't we need two thresholds for the V?
- Explain why we look for the cube in the point cloud instead of the color image
- Explain how you picked the grasp and release heights from the cube/tape's height. Why can this potentially work for taller/shorter cubes?

6 Appendix

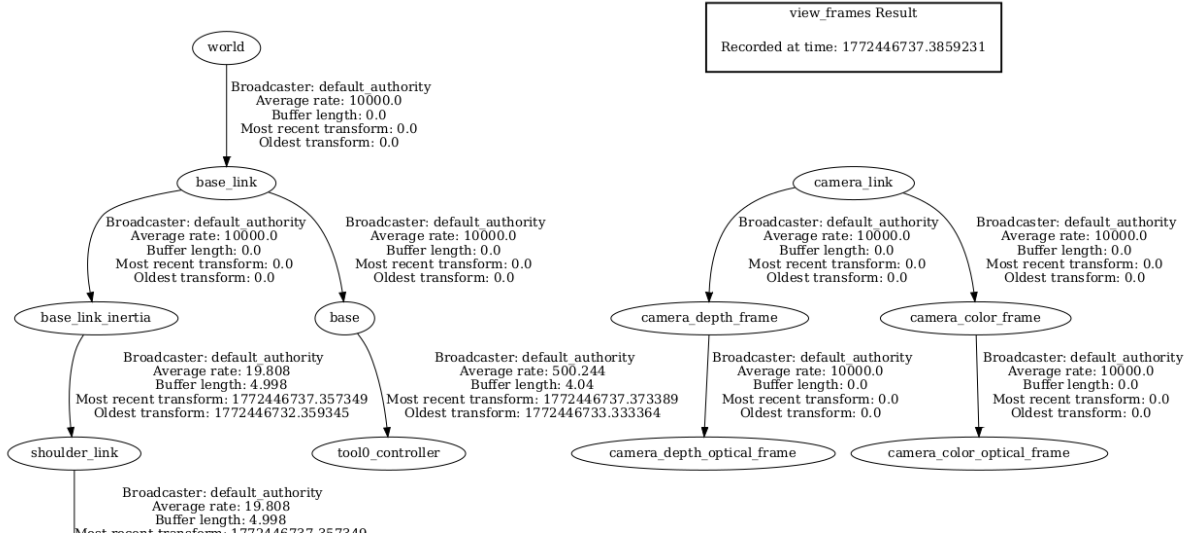


Figure 5: Disconnected Frames

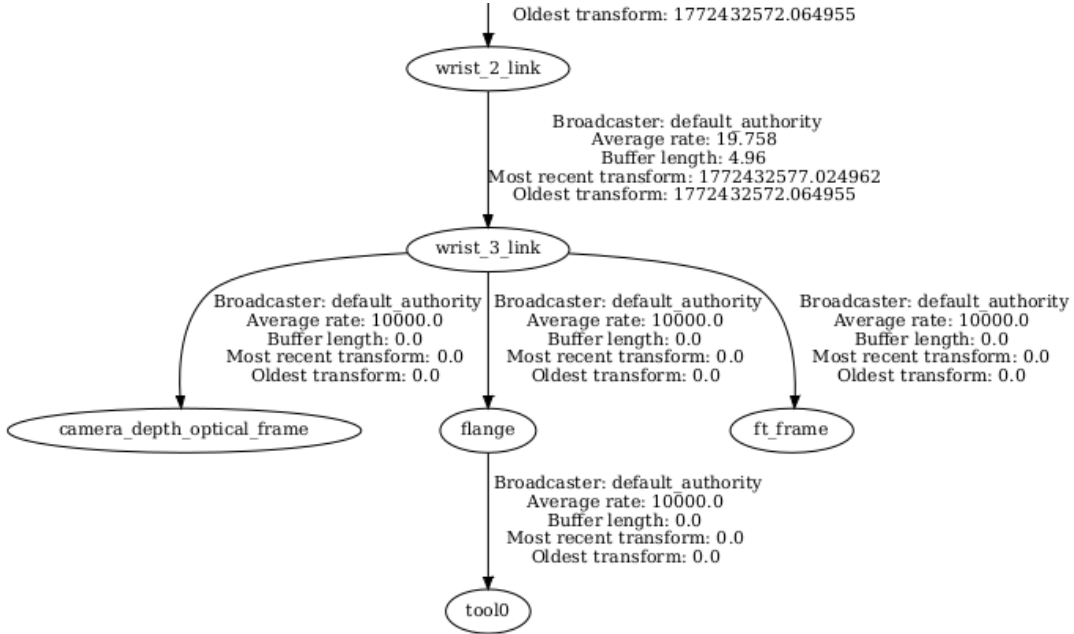


Figure 6: Connected Frames